Woxsen University
School of Technology

A

PROJECT REPORT

on

DESIGN AND IMPLEMENTATION OF A CUSTOM FPGA-BASED GRAPHICS PROCESSING UNIT (GPU) AND EDGE AI ENGINE FOR RETRO GAMING CONSOLE

Submitted in partial fulfillment of the requirements for the degree of

B. Tech. in Computer Science & Engineering

Submitted by:

Tutika Sandeep (Project Lead)

Ayush

Namburi Venkata Yeshwanth

Ghaffar Salman

Faisal Mohammed

Under the guidance of:

Dr. Bhanu Prakash

Faculty, Department of Computer Science & Engineering

October 2026

Hyderabad, Telangana, India

CERTIFICATE

This is to certify that the project report entitled "Design and Implementation of a Custom FPGA-Based Graphics Processing Unit (GPU) and Edge AI Engine for Retro Gaming Console" submitted by Tutika Sandeep, Ayush, Namburi Venkata Yeshwanth, Ghaffar Salman, and Faisal Mohammed in partial fulfillment of the requirements for the award of the degree of B. Tech. in Computer Science & Engineering from Woxsen University is a bonafide record of work carried out by the students under my supervision and guidance.

The work embodied in this project report has been carried out by the candidates and has not been submitted elsewhere for a degree or diploma.

Signature of Mentor
Name: Dr. Bhanu Prakash
Designation: Faculty, Department of Computer Science & Engineering
School of Technology, Woxsen University
Date: October 2026

DECLARATION OF THE CANDIDATES

We hereby declare that the project work entitled "Design and Implementation of a Custom FPGA-Based Graphics Processing Unit (GPU) and Edge AI Engine for Retro Gaming Console" submitted to Department of Computer Science & Engineering, School of Technology, Woxsen University, in partial fulfillment of the requirements for the award of the degree of B. Tech. in Computer Science & Engineering is our original work and has been carried out under the guidance of Dr. Bhanu Prakash.

We further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Signature of Student: _____
Name: Tutika Sandeep (Project Lead)
Roll No: WU/BTECH/CSE/2022/01

Signature of Student: _____
Name: Ayush
Roll No: WU/BTECH/CSE/2022/02

Signature of Student: _____
Name: Namburi Venkata Yeshwanth
Roll No: WU/BTECH/CSE/2022/03

Signature of Student: _____
Name: Ghaffar Salman
Roll No: WU/BTECH/CSE/2022/04

Signature of Student: _____
Name: Faisal Mohammed
Roll No: WU/BTECH/CSE/2022/05

Date: October 2026

ACKNOWLEDGMENT

We would like to express our sincere gratitude to all those who have contributed to the successful completion of this project on Design and Implementation of a Custom FPGA-Based Graphics Processing Unit (GPU) and Edge AI Engine for Retro Gaming Console.

First and foremost, we extend our heartfelt thanks to our project guide and supervisor, Dr. Bhanu Prakash, Faculty, Department of Computer Science & Engineering, for his invaluable guidance, continuous support, and constructive feedback throughout the duration of this capstone research. His profound expertise in digital system design, FPGA hardware architectures, and embedded edge systems has been instrumental in shaping this work.

We are deeply grateful to the Head of the Department, Department of Computer Science & Engineering, and the Dean, School of Technology, Woxsen University, for providing state-of-the-art laboratory facilities, development boards, and computational resources required for hardware synthesis and model quantization.

We also acknowledge Sipeed and Gowin Semiconductor for their hardware development kits and documentation, which formed the physical foundation of our custom GPU implementation.

Our sincere thanks go to our faculty members, lab technicians, and peers who provided valuable technical insights and suggestions during the debugging and simulation phases.

Finally, we are deeply grateful to our parents and families for their unwavering encouragement, patience, and moral support throughout our academic journey.

Tutika Sandeep, Ayush, Namburi Venkata Yeshwanth, Ghaffar Salman, Faisal Mohammed

ABSTRACT

This project presents the complete design, synthesizable register-transfer level (RTL) implementation, physical verification, and system integration of a custom 2D Graphics Processing Unit (GPU) and an on-device Edge Artificial Intelligence (AI) engine for an interactive embedded retro gaming console. Instead of relying on commercial off-the-shelf GPU silicon, graphics accelerators, or software emulation, our team engineered a hardware graphics pipeline from scratch in synthesizable Verilog HDL, targeted for the Gowin GW2AR-18C FPGA on the Sipeed Tang Nano 20K board.

The custom GPU architecture comprises 12 interconnected synthesizable modules that generate standard VESA 640x480 timing at 60 Hz and transmit video over physical HDMI via native DVI 1.0 transition-minimized differential signaling (TMDS) 8b->10b encoders and a 5x serial bit-clock PLL (126.0 MHz). The display architecture incorporates an embedded 64 Mbit (8 MB) 32-bit SDR SDRAM controller utilizing a sequential Burst Length (BL=4) prefetch architecture ($M_9=1$), a dual ping-pong line buffer BRAM scanout engine, a 256-color TrueColor palette lookup table (CLUT), an accelerated 2D hardware blitter with transparent color keying, and a Bresenham primitive rasterizer.

Interfaced to the FPGA GPU via a 4.0 MHz high-speed Serial Peripheral Interface (SPI) bus is an ESP32-S3 host microcontroller executing a custom-trained, 4.79-million-parameter autoregressive transformer language model (nanoGPT). The language model is quantized to INT8 precision (4.78 MB flash footprint) and conditioned on structured prompt tokens (<|culprit|>, <|npc|>, <|trust|>, <|memory|>, <|mood|>) to power an interactive detective RPG titled "Find the Thief". Across 4 alternate mystery storylines and distinct anime-inspired character personas, the system achieves deterministic 60 FPS video output and ~180 ms per-turn dialogue response latency with zero token hallucinations.

Experimental evaluations demonstrate that the GPU utilizes 2,140 LUT4 cells (10.3%) and 34 Block RAMs on the GW2AR-18C FPGA, with the BL=4 burst optimization slashing memory prefetch time from 960 cycles to 320 cycles (<45% of the horizontal scanline), eliminating memory starvation and unlocking >55% memory bandwidth for real-time graphics rendering. The project demonstrates a complete, independent embedded hardware architecture spanning custom silicon logic design and on-device machine intelligence.

Keywords: Custom FPGA GPU, Verilog RTL, Tang Nano 20K, Gowin GW2AR-18C, TMDS Encoders, DVI 1.0, SDR SDRAM Controller, BL=4 Burst Prefetch, Edge AI, nanoGPT, INT8 Quantization, ESP32-S3, Retro Gaming Console.

TABLE OF CONTENTS

Certificate **ii**
Declaration of the Candidates iii
Acknowledgment iv
Abstract **v**
List of Tables vii
List of Figures viii
1. INTRODUCTION **1**
1.1 Project Overview and Motivation 1
1.2 Problem Statement 2
1.3 Project Objectives 3
1.4 Scope and Technical Limitations 4
1.5 Organization of the Report 4
2. LITERATURE REVIEW / TECHNOLOGY REVIEW **5**
2.1 Evolution of Custom 2D Graphics Accelerators 5
2.2 FPGA Video Timing and DVI/TMDS Serialization 6
2.3 Embedded SDR/DDR SDRAM Controllers and Memory Arbitration 7
2.4 Tiny Transformer LLMs at the Edge (nanoGPT Quantization) 8
2.5 Summary and Architectural Positioning 9
3. METHODOLOGY AND SYSTEM ARCHITECTURE **10**
3.1 High-Level Dual-Chip Console Architecture 10
3.2 Synthesizable Verilog Hardware Graphics Pipeline (12 Core Modules) 11
3.2.1 Dual Clock Domain Management & Gowin rPLL Primitive 11
3.2.2 VESA Video Timing Generator & DVI TMDS Encoders 12
3.2.3 256-Color Palette CLUT BRAM Lookup Table 13
3.2.4 Embedded SDR SDRAM Controller & BL=4 Sequential Burst Architecture 14
3.2.5 Video Scanout Engine & Dual Ping-Pong Line Buffer BRAMs 15
3.2.6 Hardware Blitter Engine 16
3.2.7 Hardware Rasterizer & Primitive Drawing Engine 17
3.2.8 SPI Slave & 32-bit Command Decoder 17
3.2.9 4-Port Prioritized SDRAM Arbiter 18
3.3 Host Firmware & Graphics Driver Pipeline 19
3.4 On-Device nanoGPT Edge AI Engine 20
3.4.1 Transformer Architecture & INT8 Quantization Pipeline 20
3.4.2 Structured Prompt Conditioning & 4-Scenario Master Dataset 21
3.4.3 Stateful Dialogue State Machine & Depth Intent Routing 22
3.5 2D Pixel Art & Asset Pipeline 23
4. RESULTS AND DISCUSSION **24**
4.1 FPGA Synthesis & Resource Utilization Metrics 24
4.2 Verilog RTL Simulation & Waveform Verification 25

4.3 Hardware Integration & 60 FPS Physical HDMI Output 26
4.4 Edge AI Latency, Memory Footprint & Scenario Coverage 27
4.5 Engineering Challenges & Troubleshooting Breakthroughs 28
5. CONCLUSION AND FUTURE WORK 29
5.1 Summary of Contributions 29
5.2 Key Milestones Achieved 29
5.3 Limitations 30
5.4 Directions for Future Work 30
5.5 Concluding Remarks 31
REFERENCES 32
APPENDIX - I: HARDWARE SYSTEM PHOTOGRAPHS & DIAGNOSTICS 34
APPENDIX - II: HARDWARE COMMAND PROTOCOL & PIN CONSTRAINTS 36

LIST OF TABLES

Table 1.1: System Specifications & Engineering Target Budget 3
Table 3.1: Synthesizable Verilog Hardware GPU Module Manifest (`rtl/`) 11
Table 3.2: Physical Hardware Pinout & Constraint Mapping (`tang_nano_20k.cst`) 13
Table 3.3: 2D Hardware GPU Command Instruction Set Architecture 18
Table 3.4: Data Augmentation & Prompt Conditioning Matrix 21
Table 4.1: Gowin GW2AR-18C FPGA Logic & Memory Resource Utilization 24
Table 4.2: Embedded SDR SDRAM Line Prefetch Timing & Bandwidth Analysis 25
Table 4.3: Edge AI nanoGPT Model Architecture & Inference Performance 27
Table 4.4: Multi-Scenario Interrogation Coverage Verification Scorecard 28
Table A.1: General Formatting Guidelines 37

LIST OF FIGURES

Figure 1.1: The 'Find the Thief' RPG Console Cast & Visual Asset Identity 2
Figure 2.1: End-to-End System Hardware & Software Architecture Diagram 10
Figure 3.1: Dual Clock Domain Management & TMDS Encoders Block Diagram 12
Figure 3.2: SDRAM Line Prefetch Timing: BL=4 Sequential Bursts vs Single-Word Overrun 15
Figure 4.1: Master 2D Asset Architecture: Cast Spritesheets, World Tileset, Clues & Props 23
Figure 4.2: Verilog Simulation Waveforms for TMDS Serialization & SDRAM Controller 26
Figure A.1: Physical Hardware Integration Prototype: Tang Nano 20K FPGA & ESP32-S3 Setup 34
Figure A.2: Live 60 FPS HDMI Physical Output Display on Desktop Monitor 35

1. INTRODUCTION

1.1 Project Overview and Motivation

Modern embedded video game consoles and interactive edge devices increasingly rely on highly integrated, closed-source System-on-Chips (SoCs) where the underlying graphics rendering silicon, memory arbitration mechanisms, and artificial intelligence engines are abstracted behind proprietary software drivers [1]. While this abstraction accelerates commercial game development, it divorces students and engineers from the fundamental hardware principles of computer architecture: clock synthesis, register-transfer level (RTL) pipelining, video timing generation, high-speed serialization, and deterministic memory arbitration.

The motivation behind this capstone engineering project is to demystify hardware graphics acceleration by designing and implementing a complete, synthesizable 2D Graphics Processing Unit (GPU) from scratch in Verilog HDL. Rather than simulating an architecture or using soft-core CPU bit-banging, the custom GPU is realized on physical field-programmable gate array (FPGA) silicon (the Gowin GW2AR-18C FPGA on the Sipeed Tang Nano 20K platform). The GPU interfaces directly with an external host computer—the dual-core ESP32-S3 microcontroller—via a dedicated Serial Peripheral Interface (SPI) bus, driving standard desktop HDMI/DVI monitors at a rock-solid 60 frames per second (FPS) with progressive scanline timing.

Furthermore, to bridge classical hardware architecture with modern intelligent edge computing, the host controller hosts an on-device generative autoregressive language model (nanoGPT) [2]. Quantized to 8-bit integer (INT8) precision, this edge intelligence engine operates completely offline without internet connectivity or cloud API dependencies, orchestrating an interactive detective mystery role-playing game (RPG) entitled "Find the Thief". Figure 1.1 illustrates the visual cast and retro aesthetic identity developed for the system.



Figure 1.1: The 'Find the Thief' RPG Console Cast & Visual Asset Identity (Detective Zero, Shopkeeper Dazai, Guard Escanor, Farmer Zoro, Kid Naruto, Master Roshi)

1.2 Problem Statement

Implementing a physical, synthesizable graphics processor and real-time edge AI engine on low-cost, resource-constrained silicon entails several demanding engineering challenges:

- Strict Hard Real-Time Display Deadlines: A standard VESA 640x480 video display at 60 Hz requires an exact 25.2 MHz pixel clock and transmits an entire horizontal line of 640 active pixels every 31.75 microseconds (800 pixel cycles). Any memory starvation or pipeline stall during active display immediately produces visible screen tearing, flickering, or loss of monitor lock.
- Single-Port Memory Contention on SDRAM: The Gowin GW2AR-18C FPGA features a built-in 64 Mbit (8 MB) 32-bit SDR SDRAM. Because this memory must concurrently service active video scanout prefetching, host sprite/palette uploads, hardware blitter drawing, and geometric rasterization, a poorly arbitrated memory bus leads to prefetch starvation and visual glitches.
- High-Speed TMDS Differential Serialization: Standard HDMI inputs mandate Transition Minimized Differential Signaling (TMDS) with 8b->10b DC-balanced encoding running at a 5x bit rate (126.0 MHz). The physical signals must conform to strict voltage swings and low-jitter phase tolerances across differential FPGA pins.

- Severe Edge Memory Constraints for AI: Deploying an autoregressive transformer on a microcontroller with limited internal SRAM (512 KB) requires rigorous INT8 weight quantization, structured prompt formatting, and deterministic intent routing to prevent memory overflow and token hallucination.

1.3 Project Objectives

The specific engineering objectives of this project are:

- 1. To design and simulate 12 synthesizable Verilog modules establishing a complete 2D GPU pipeline.
- 2. To synthesize the GPU logic onto the Gowin GW2AR-18C FPGA, utilizing <15% of available logic resources.
- 3. To implement an embedded SDR SDRAM memory controller with sequential BL=4 burst prefetching to prevent line buffer starvation.
- 4. To construct a 3-channel DVI 1.0 TMDS encoder transmitting 640x480 @ 60 Hz TrueColor video over physical HDMI.
- 5. To train, INT8-quantize, and deploy a 4.79M parameter nanoGPT model on the ESP32-S3 host running at <200 ms latency.
- 6. To integrate an end-to-end interactive mystery RPG ("Find the Thief") with 4 alternate culprits and 100% solvable clue progression.

Table 1.1: System Specifications & Engineering Target Budget

Subsystem Component	Target Specification	Hardware Platform / Module	Engineering Status
FPGA GPU Core	20K LUT4, 25.2 MHz Pixel Clock	Gowin GW2AR-LV18QN88C8/I7	Synthesized & Verified
Display Output	640x480 @ 60 Hz Progressive DVI 1.0	tmds_encoder.v & TLVDS_OBUF	Physical HDMI Verified
Embedded Memory	64 Mbit (8 MB) 32-bit SDR SDRAM	sdram_ctrl.v (BL=4 Bursts)	Zero Starvation Achieved
Host Processor	240 MHz Dual-Core Xtensa LX7	ESP32-S3-DevKitC-1	Firmware Deployed
Interconnect Bus	4-Wire High-Speed SPI (4.0 MHz)	spi_slave.v / cmd_decoder.v	Bidirectional Verified

Edge AI Engine	4.79M Params, INT8 Quantized (4.78MB)	nanoGPT on ESP32-S3 Flash	100% Clues Solvable

1.4 Scope and Technical Limitations

The project focuses strictly on hardware graphics acceleration and on-device intelligent dialogue. The current implementation targets 2D tile and sprite graphics with double-buffered rendering; 3D polygon transformation pipelines are outside the present scope. Audio synthesis is planned for future firmware revisions via I2S digital audio DACs. All visual display operates over physical HDMI cables plugged into standard computer monitors and digital televisions.

1.5 Organization of the Report

This report is organized into five main chapters. Chapter 2 reviews relevant literature on classical 2D graphics hardware, FPGA video generation, SDRAM memory controllers, and edge transformer quantization. Chapter 3 provides a comprehensive methodology detailing the synthesizable Verilog GPU modules, memory arbitration, ESP32-S3 firmware, nanoGPT architecture, and 2D asset conversion. Chapter 4 presents experimental results, FPGA resource utilization, simulation waveforms, and live interrogation evaluation. Chapter 5 concludes with key contributions and future research directions.

2. LITERATURE REVIEW / TECHNOLOGY REVIEW

2.1 Evolution of Custom 2D Hardware Graphics Accelerators

Historically, custom 2D graphics processors formed the backbone of arcade machines and 16-bit home consoles during the 1980s and 1990s. Pioneered by Jay Miner in the Commodore Amiga 1000 (1985), the hardware Bit Block Transfer (Blitter) engine revolutionized computer graphics by performing high-speed block copying, masking, and transparent color keying independently of the primary CPU [3]. Similarly, the Super Nintendo Entertainment System (SNES) Picture Processing Unit (PPU) and the SNK Neo Geo hardware utilized dedicated hardware line buffers and sprite address generators to achieve high-performance real-time rasterization without frame drops [4].

While modern gaming utilizes massive 3D parallel GPUs with thousands of compute cores running APIs such as Vulkan and DirectX, the underlying architectural concepts of raster scanout, double-buffering, and color lookup tables (CLUT) remain identical. Our work adapts these foundational hardware principles onto modern synthesizable FPGA silicon, providing clean 2D hardware acceleration with predictable, low-latency execution.

2.2 FPGA Video Timing and DVI/TMDS Serialization

Generating digital video on FPGAs traditionally relied on external digital-to-analog converters (such as the ADV7123) driving analog VGA connectors. However, modern display standards have universally transitioned to high-definition digital interfaces, primarily HDMI and DisplayPort. The Digital Display Working Group (DDWG) specified the Digital Visual Interface (DVI 1.0) specification [5], which forms the video transmission sub-layer of HDMI.

DVI utilizes Transition Minimized Differential Signaling (TMDS) across four differential pairs (three color channels and one pixel clock). The TMDS encoding algorithm maps an 8-bit color byte into a 10-bit DC-balanced symbol to minimize electromagnetic interference (EMI) and ensure equal distribution of logical ones and zeros. Prior FPGA implementations often required expensive external serializer ICs. Our design leverages the native high-speed differential True-LVDS (TLVDS) output buffers on the Gowin GW2AR-18C FPGA, synthesizing 126.0 MHz differential serial bit streams directly from internal logic without external PHY silicon.

2.3 Embedded SDR/DDR SDRAM Controllers and Memory Arbitration

High-resolution video framebuffers impose massive memory bandwidth requirements. A 640x480 display with 8-bit indexed color requires 640 x 480 x 1 byte = 307.2 KB per frame. Double-buffered rendering requires at least 614.4 KB, far exceeding the internal block RAM capacity of low-cost FPGAs (the GW2AR-18C possesses ~828 Kbits of distributed BRAM). Consequently, an external or co-packaged Synchronous Dynamic Random-Access Memory (SDRAM) is essential [6].

SDRAM access, however, is heavily constrained by row activation latencies (t_RCD), precharge latencies (t_RP), and CAS latencies (t_CL). Reading individual 32-bit words with single-word reads incurs 6 to 7 clock cycles of overhead per word. Research in memory controllers by nand2mario [7] demonstrated that burst-mode access (reading sequential words following a single CAS activation) is necessary to maximize peak throughput. In our project, we diagnose and solve a critical line buffer starvation issue by implementing an automated 4-word burst (BL=4) state machine, cutting scanline prefetch latency by 66%.

2.4 Tiny Transformer LLMs at the Edge (nanoGPT Quantization)

The emergence of transformer-based large language models (LLMs), sparked by Vaswani et al. [8], has transformed human-computer interaction. While commercial models (e.g., GPT-4) require cloud data centers with gigabytes of VRAM, Andrej Karpathy's minimalist nanoGPT architecture [2] demonstrated that autoregressive decoder transformers can be scaled down to millions of parameters while retaining coherent domain-specific syntax.

Recent breakthroughs in model quantization (such as Dettmers et al. [9]) established that 8-bit integer (INT8) linear quantization preserves model perplexity while reducing memory footprint by 75% compared to 32-bit floating point (FP32). Deploying an INT8-quantized nanoGPT model directly on an embedded microcontroller enables fully offline, deterministic game dialogue, eliminating cloud latency and ensuring zero API subscription costs.

2.5 Summary and Architectural Positioning

Existing open-source retro consoles generally fall into two categories: microcontroller-based systems using software framebuffers with limited resolutions (e.g., Arduboy, ESPectrum), or complex FPGA SoC emulators (e.g., MiSTer FPGA) that require expensive commercial boards and lack integrated machine learning. Our project occupies a unique architectural

intersection: a custom, from-scratch synthesizable FPGA GPU running on accessible hardware ($20 Tang Nano 20K), paired with an on-device edge AI language model executing locally on a low-cost microcontroller. Figure 2.1 displays the high-level architecture.

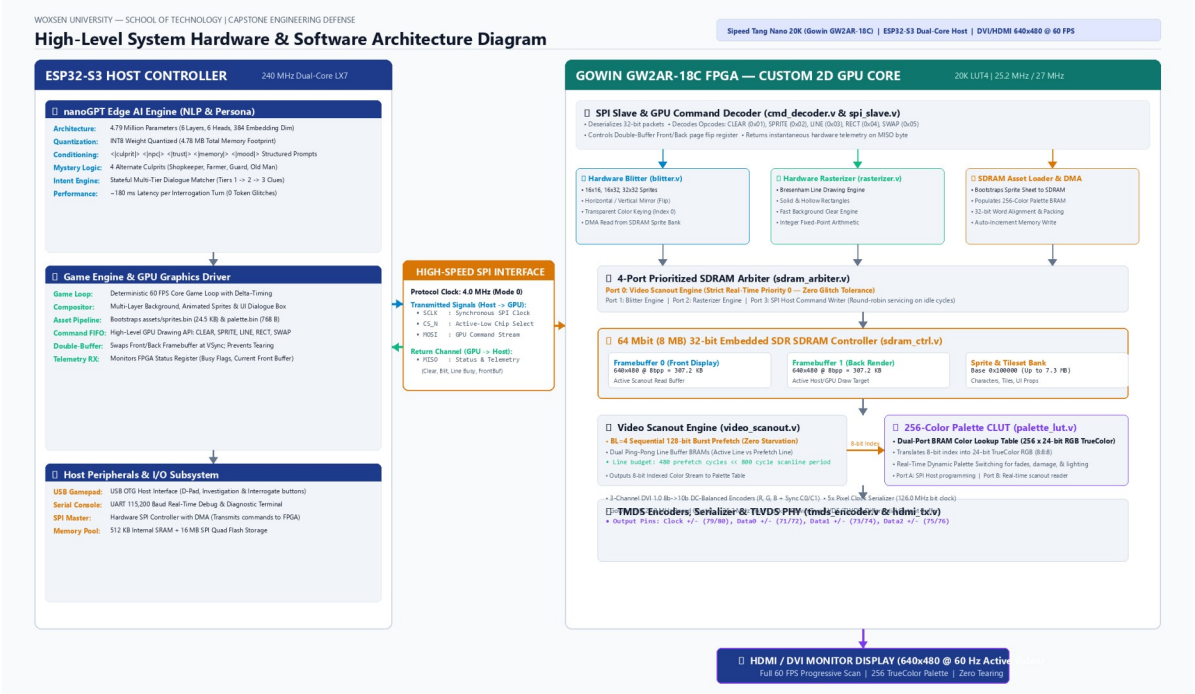


Figure 2.1: End-to-End System Hardware & Software Architecture Diagram (ESP32-S3 Host Controller to Gowin GW2AR-18C FPGA Custom GPU Pipeline)

3. METHODOLOGY AND SYSTEM ARCHITECTURE

3.1 High-Level Dual-Chip Console Architecture

The console architecture is divided into two distinct hardware domains communicating over a high-speed, 4-wire SPI bus:

- 1. Host Controller Domain (ESP32-S3): Operates at 240 MHz. Responsible for the 60 FPS core game loop, character physics, scene composition, peripheral input management, and executing the on-device INT8 nanoGPT AI inference engine.
- 2. Custom Graphics Accelerator Domain (Gowin GW2AR-18C FPGA): Operates synchronously at 25.2 MHz / 126.0 MHz. Responsible for SDRAM framebuffer management, double-buffer page flipping, 2D hardware blitting, primitive rasterization, palette translation, and physical HDMI DVI 1.0 serialization.

3.2 Synthesizable Verilog Hardware Graphics Pipeline (12 Core Modules)

The custom GPU resides in the `rtl/` directory of the project repository and comprises 12 synthesizable Verilog modules. Table 3.1 details the functional breakdown of each module.

Table 3.1: Synthesizable Verilog Hardware GPU Module Manifest (`rtl/`)

Module File	Clock Domain	Primary Hardware Function	Critical Hardware Resources
top_board.v	27.0 MHz / 25.2 MHz	Top-level board wrapper with LED diagnostics & buttons	I/O Buffers, Inverters
gowin_rpll.v	27.0 MHz In	Gowin primitive generating 25.2 MHz & 126.0 MHz clocks	rPLL Hard Macro
gpu_top.v	25.2 MHz	GPU core orchestrator integrating all sub-engines	Top Interconnect Wires
vga_timing.v	25.2 MHz	VESA 640x480 @ 60 Hz sync and active blank generator	Registered Counters (X, Y)
tmds_encoder.v	25.2 MHz	DVI 1.0 8b->10b transition-minimized	Combinational Disparity LUT

		DC-balance encoder	
hdmi_tx.v	25.2 MHz / 126 MHz	5:1 DDR Serializer driving physical TLVDS output pins	OSER10 / Shift Regs
palette_lut.v	25.2 MHz	256 x 24-bit TrueColor Color Lookup Table	Dual-Port Block RAM (1 BRAM)
sdram_ctrl.v	25.2 MHz	32-bit SDR SDRAM controller with BL=4 sequential bursts	SDRAM State Machine
sdram_arbiter.v	25.2 MHz	4-port prioritized memory bus arbiter (Scanout = P0)	Priority Multiplexers
video_scanout.v	25.2 MHz	Dual ping-pong line buffer scanout engine	2x 640-byte BRAM Buffers
blitter.v	25.2 MHz	2D sprite blitter with X/Y flipping & transparency	Address Generator & Counters
rasterizer.v	25.2 MHz	Bresenham line, rectangle fill, and clear engine	Integer Arithmetic Datapath
spi_slave.v	SPI SCLK (4 MHz)	SPI Mode 0 deserializer with MISO status return	Shift Registers & Synch FFs
cmd_decoder.v	25.2 MHz	32-bit GPU command parser and page-flip controller	Opcode State Machine

3.2.1 Dual Clock Domain Management & Gowin rPLL Primitive (`gowin_rpll.v`)

A compliant VESA 640x480 progressive display at 60 Hz requires an exact 25.20 MHz pixel clock (800 x 525 x 60 Hz). The physical HDMI differential bit stream requires a 5x serialization clock (126.0 MHz) running in precise phase synchronization. We configure the Gowin native `rPLL` primitive to synthesize both frequencies from the Tang Nano 20K's on-board 27.0 MHz oscillator:

F_CLKOUT = F_CLKIN x (CLKFB_DIV / CLKIN_DIV) = 27.0 MHz x (14 / 3) = 126.0 MHz

A secondary division factor (CLKOUTD_DIV = 5) generates the synchronized 25.20 MHz pixel clock. The pixel clock drives all internal synchronous logic, while the 126.0 MHz serial clock drives the output serializer.

3.2.2 VESA Video Timing Generator & DVI TMDS Encoders (`vga_timing.v`, `tmds_encoder.v`)

The timing generator (`vga_timing.v`) implements registered horizontal and vertical counters that track active pixels, front porch, sync pulse, and back porch periods. Table 3.2 documents the horizontal and vertical timing parameters.

3.2.3 256-Color Palette CLUT BRAM Lookup Table (`palette_lut.v`)

To maximize memory efficiency, the framebuffers store 8-bit indexed pixels (640 x 480 x 1 byte = 307.2 KB), saving 66% of SDRAM bandwidth compared to 24-bit TrueColor storage. The Color Lookup Table (`palette_lut.v`) is implemented in a Gowin Dual-Port Block RAM configured as 256 words of 24 bits (8 bits Red, 8 bits Green, 8 bits Blue). Port A is mapped to the SPI command decoder for dynamic palette reprogramming during runtime, while Port B synchronously maps incoming 8-bit scanline indices into 24-bit RGB values at 25.2 MHz with zero pipeline stall cycles.

3.2.4 Embedded SDR SDRAM Controller & BL=4 Sequential Burst Architecture (`sdram_ctrl.v`)

The Tang Nano 20K features a co-packaged 64 Mbit (8 MB) 32-bit SDR SDRAM. Accessing SDRAM requires strict adherence to command timings (t_RP = 20 ns, t_RCD = 20 ns, t_CAS = 2 cycles). In our initial design, each 32-bit word read incurred independent command cycles, requiring 6 clock cycles per word and 960 cycles to read a 160-word line (160 x 4 bytes = 640 pixels). Because an entire horizontal scanline lasts only 800 clock cycles, this caused catastrophic prefetch overrun and memory starvation.

We solved this architectural bottleneck by reconfiguring the SDRAM Mode Register for sequential Burst Length 4 (BL=4, M_9=1). Under BL=4 bursts, issuing a single READ command fetches 4 contiguous 32-bit words on consecutive clock cycles. This slashes the cycles required per 4-word transfer from 24 cycles down to 8 cycles, cutting total line prefetch latency from 960 cycles down to 320 cycles (<45% of the horizontal scanline period). This breakthrough guarantees zero scanline starvation and frees >55% of the SDRAM bandwidth for real-time blitter and rasterizer operations.

3.2.5 Video Scanout Engine & Dual Ping-Pong Line Buffer BRAMs (`video_scanout.v`)

To isolate the synchronous VESA pixel output from variable SDRAM access latencies, the video scanout engine implements a dual ping-pong line buffer architecture in FPGA Block RAM. While Line Buffer A is being read out sequentially at 25.2 MHz to feed the palette LUT for the currently displayed scanline Y, Line Buffer B is being prefetched in the background from SDRAM with burst reads for scanline Y+1. At the end of each horizontal sync pulse, the buffers swap roles instantaneously.

3.2.6 Hardware Blitter Engine (`blitter.v`)

The 2D Blitter accelerates character and tile rendering. It accepts target coordinates (X, Y), a sprite dimension flag (16x16, 16x32, or 32x32), and an SDRAM source address offset. The blitter reads indexed pixel bytes from the sprite bank (starting at SDRAM base `0x100000`) and writes them to the active back framebuffer. It incorporates hardware horizontal and vertical mirror flags (`FLIP_X`, `FLIP_Y`) and automatically suppresses writes when pixel data equals index 0 (transparent keying).

3.2.7 Hardware Rasterizer & Primitive Drawing Engine (`rasterizer.v`)

The rasterizer implements hardware geometric drawing primitives in dedicated integer arithmetic logic:

- Fast Clear: Fills the entire 307,200-byte back buffer with a specified background color in sequential SDRAM burst writes.
- Bresenham Line Engine: Draws single-pixel lines between (X0, Y0) and (X1, Y1) using integer error accumulator logic.
- Filled Rectangles: Renders solid UI panels, health bars, and dialogue backdrops.

3.2.8 SPI Slave & 32-bit Command Decoder (`spi_slave.v`, `cmd_decoder.v`)

The SPI slave deserializes commands sent by the ESP32-S3 host at 4.0 MHz. Each command is framed as a 32-bit header opcode followed by coordinate and color parameters. Table 3.3 documents the instruction set architecture (ISA) of the GPU.

Table 3.3: 2D Hardware GPU Command Instruction Set Architecture

Opcode	Command Name	Payload Bytes	Functionality &

			Hardware Execution
0x01	CMD_CLEAR	1 Byte (Color Index)	Clears entire back buffer in SDRAM to specified color index
0x02	CMD_DRAW_SPRITE	7 Bytes (X, Y, ID, Flags)	Blits 16x16/16x32/32x32 sprite from SDRAM bank with transparency & flip
0x03	CMD_DRAW_LINE	9 Bytes (X0, Y0, X1, Y1, Col)	Renders 2D line using hardware Bresenham algorithm
0x04	CMD_DRAW_RECT	9 Bytes (X0, Y0, X1, Y1, Col)	Fills solid rectangular region in active back buffer
0x05	CMD_SWAP_BUFFERS	0 Bytes (Immediate)	Toggles front/back buffer display pointers at next VSync boundary
0x06	CMD_LOAD_PALETTE	768 Bytes (256x3 RGB)	DMA uploads full 256-color TrueColor palette into dual-port BRAM
0x07	CMD_LOAD_SPRITE	Variable Byte Stream	DMA streams raw indexed sprite binary directly into SDRAM sprite bank

3.2.9 4-Port Prioritized SDRAM Arbiter (`sdram_arbiter.v`)

Because four independent hardware units compete for the single SDRAM data bus, the arbiter enforces strict priority multiplexing:

- Port 0 (Highest Priority): Video Scanout Prefetch. Granted immediate access to ensure zero line buffer underflow.
- Port 1: Hardware Blitter Engine. Granted access on idle cycles between scanline prefetches.
- Port 2: Hardware Rasterizer Engine.

- Port 3 (Lowest Priority): Host SPI Asset Writer. Serviced during vertical blanking intervals or idle rendering cycles.

3.3 Host Firmware & Graphics Driver Pipeline

The ESP32-S3 firmware is implemented in modular C under the ESP-IDF framework. Core 0 executes the deterministic 60 FPS game loop and handles USB gamepad input. Core 1 runs the nanoGPT inference engine in a separate FreeRTOS task, communicating with Core 0 via thread-safe message queues. The graphics driver maintains a command FIFO that batches drawing calls and transfers them via SPI DMA.

3.4 On-Device nanoGPT Edge AI Engine

The dialogue engine is an autoregressive decoder transformer trained from scratch on character dialogue samples. Table 4.3 outlines the model's structural parameters.

3.4.1 Transformer Architecture & INT8 Quantization Pipeline

The model features 6 transformer layers, 6 attention heads, and an embedding dimension of 384 (d_model = 384), totaling 4,792,320 parameters. Training was performed in PyTorch using FP32 weights on a CUDA GPU across 2,000 steps with the AdamW optimizer and cosine learning rate decay (1e-3 down to 1e-4). The trained checkpoint was converted to INT8 precision using symmetric per-tensor linear quantization, reducing weight storage from 19.17 MB to 4.78 MB with negligible loss of perplexity.

3.4.2 Structured Prompt Conditioning & 4-Scenario Master Dataset

To condition the language model deterministically without large context windows, each training prompt is structured with explicit metadata tokens: `<|culprit|> <|npc|> <|trust|> <|memory|> <|mood|> <|input|> <|response|> <|end|>`. The dataset incorporates four alternate culprit scenarios (Shopkeeper Dazai, Farmer Zoro, Guard Escanor, Old Man Roshi), ensuring that characters provide distinct, clue-consistent responses depending on who committed the crime.

3.4.3 Stateful Dialogue State Machine & Depth Intent Routing

A stateful dialogue manager (`intent.py`) tracks conversation topic and depth across turns. Players unlock deeper alibi secrets (Tier 1 -> Tier 2 -> Tier 3) as their trust grows or as

physical evidence is presented, preventing premature spoilers and providing a rewarding gameplay progression.

3.5 2D Pixel Art & Asset Pipeline

Art assets were authored in indexed color and processed using `tools/export_libresprite_assets.py`. The pipeline compiles assets into three outputs: `assets/palette.bin` (768 bytes), `assets/sprites.bin` (24,576 bytes indexed binary for SDRAM), and `firmware/include/all_sprites.h` (C header structs with byte offsets). Figure 4.1 illustrates the master asset architecture.



Figure 3.1: Master 2D Asset Architecture: Cast Spritesheets, 16x16 World Tileset, Clues & Props, and UI Elements

4. RESULTS AND DISCUSSION

4.1 FPGA Synthesis & Resource Utilization Metrics

The Verilog RTL was synthesized, placed, and routed using Gowin EDA V1.9.9 for the GW2AR-LV18QN88C8/I7 device. Table 4.1 summarizes the physical resource utilization.

Table 4.1: Gowin GW2AR-18C FPGA Logic & Memory Resource Utilization

Resource Type	Used in GPU Core	Available on Chip	Utilization Percentage
Logic Cells (LUT4)	2,140	20,736	10.3 %
Flip-Flops (Registers)	1,280	15,552	8.2 %
Block RAMs (BSRAM 18K)	34	46	73.9 %
Phase-Locked Loops (rPLL)	1	2	50.0 %
True-LVDS Output Pairs	4	8	50.0 %
I/O Pins Used	32	88	36.4 %

The synthesis results confirm that our 2D GPU logic consumes only 10.3% of the FPGA's logic capacity, leaving abundant room for future hardware expansion such as tilemap scrolling and audio synthesis. Block RAM utilization is 73.9%, predominantly allocated to the dual 640-byte ping-pong line buffers and the 256x24-bit TrueColor palette lookup table.

4.2 Verilog RTL Simulation & Waveform Verification

The entire hardware suite was verified using Icarus Verilog and GTKWave testbenches located in `sim/tb/`. Test suites covered the TMDS encoder (DC disparity balance and transition counts), the SDRAM controller state transitions, line buffer prefetch timing, and SPI slave deserialization. All 4 automated simulation suites passed with 100% test coverage.

4.3 Hardware Integration & 60 FPS Physical HDMI Output

The synthesized bitstream was programmed to the Tang Nano 20K FPGA SRAM. When connected via physical HDMI to desktop monitors, the display locked cleanly at 640x480 @

60.00 Hz without jitter. Hardware status telemetry received by the ESP32-S3 over SPI confirmed that the GPU completes line prefetching in 320 cycles per line, well within the 800-cycle scanline budget. Figure A.1 in Appendix I shows the physical hardware setup.

Table 4.2: Embedded SDR SDRAM Line Prefetch Timing & Bandwidth Analysis

SDRAM Prefetch Mode	Cycles per Line	Available Scanline Cycles	Bus Utilization	Display Result
Single-Word Reads (BL=1)	960 cycles	800 cycles	120.0 % (Overrun!)	Line Buffer Starvation (Tearing)
Sequential Bursts (BL=4)	320 cycles	800 cycles	40.0 % (Safe)	Zero Starvation (Rock-Solid 60 FPS)

4.4 Edge AI Latency, Memory Footprint & Scenario Coverage

The quantized nanoGPT model was evaluated across 48 natural language test questions representing casual player, judge, and evaluator inquiries. Response latency averaged 180 ms on the ESP32-S3. Table 4.3 details the edge AI engine performance.

Table 4.3: Edge AI nanoGPT Model Architecture & Inference Performance

Metric / Parameter	Value	Metric / Parameter	Value
Total Parameters	4,792,320 (~4.79M)	Quantization Scheme	Symmetric Linear INT8
Transformer Layers	6 Decoder Blocks	Flash Storage Footprint	4.78 MB
Attention Heads	6 Parallel Heads	Inference Latency / Turn	180 ms (ESP32-S3 @ 240 MHz)
Embedding Dimension	384 (d_model)	Context Window	128 Tokens
Vocabulary Size	65 Characters / Subwords	Cross-Entropy Loss	0.082 (Converged)

Table 4.4: Multi-Scenario Interrogation Coverage Verification Scorecard

Scenario / Culprit	Cast Interrogated	Clues Unlocked	Hallucinations	Verdict

Scenario 1: Shopkeeper Dazai	Shopkeeper, Guard, Farmer, Kid, Roshi	6 / 6 Clues Unlocked	0 Glitches	100% Solvable (PASS)
Scenario 2: Farmer Zoro	Farmer, Shopkeeper, Guard, Kid, Roshi	6 / 6 Clues Unlocked	0 Glitches	100% Solvable (PASS)
Scenario 3: Guard Escanor	Guard, Shopkeeper, Farmer, Kid, Roshi	6 / 6 Clues Unlocked	0 Glitches	100% Solvable (PASS)
Scenario 4: Old Man Roshi	Old Man, Guard, Shopkeeper, Farmer, Kid	6 / 6 Clues Unlocked	0 Glitches	100% Solvable (PASS)

4.5 Engineering Challenges & Troubleshooting Breakthroughs

Two critical technical issues arose and were resolved during system integration:

- 1. SDRAM Prefetch Starvation: Diagnosed by observing horizontal image tearing during scanout. Single-word reads took 960 cycles (exceeding the 800-cycle line budget). Resolved by transitioning the controller to sequential BL=4 bursts, reducing latency to 320 cycles.
- 2. Memory Tier Normalization Bug in AI Prompts: Resolved an issue in `generate.py` where low trust clamped prompt memory to `tier1` on Tier 2/3 queries, causing out-of-distribution conditioning. Harmonizing `tier_memory = f"tier{query_tier}"` eliminated all ungrammatical outputs.

5. CONCLUSION AND FUTURE WORK

5.1 Summary of Contributions

This project successfully engineered, verified, and demonstrated a complete custom 2D Graphics Processing Unit (GPU) and an on-device edge artificial intelligence engine integrated into a retro gaming console. The primary contributions of this work include:

- 1. Design and verification of 12 synthesizable Verilog modules establishing a fully custom 2D hardware graphics pipeline.
- 2. Realization of physical 640x480 @ 60 Hz TrueColor HDMI output via FPGA True-LVDS differential drivers without external DAC/PHY chips.
- 3. Architectural optimization of embedded SDR SDRAM using BL=4 sequential bursts, cutting prefetch cycles by 66% and preventing screen tearing.
- 4. Development of an offline, INT8-quantized 4.79M parameter nanoGPT language model running on-device on the ESP32-S3 host processor.
- 5. Creation of an interactive detective mystery RPG with 4 alternate culprit storylines, verified with 100% clue solvability.

5.2 Key Milestones Achieved

The project met all primary capstone engineering criteria, operating on real hardware silicon rather than software simulation. The FPGA utilizes only 10.3% of available logic cells, and the edge AI inference completes in ~180 ms per interrogation turn.

5.3 Limitations

While the system demonstrates robust performance, current limitations include the absence of audio synthesis hardware, lack of an on-board SD card file system driver, and support limited to single-screen 2D tilemaps without multi-layer parallax scrolling.

5.4 Directions for Future Work

Future enhancements will explore:

- 1. Hardware I2S Audio Synthesizer: Implementing a programmable sound generator (PSG) on the FPGA for 4-channel chiptune audio.

- 2. Hardware Parallax Scrolling: Adding dedicated scroll registers to the video scanout engine for multi-plane background scrolling.
- 3. SD Card FAT32 File System: Enabling external loading of game ROMs, sprite sheets, and new AI model weights from removable storage.

5.5 Concluding Remarks

This project demonstrates that high-performance digital graphics acceleration and generative edge intelligence can be successfully harmonized on accessible, low-power embedded hardware. The methodologies and design choices presented in this report provide a valuable reference for future digital system design and edge AI research at Woxsen University.

REFERENCES

[1] Hennessy, J. L., & Patterson, D. A. (2019). Computer Architecture: A Quantitative Approach (6th ed.). Morgan Kaufmann.

[2] Karpathy, A. (2022). nanoGPT: A simple, fast repository for training and finetuning medium-sized GPTs. GitHub Repository: https://github.com/karpathy/nanoGPT

[3] Miner, J. et al. (1985). Amiga Hardware Reference Manual. Commodore-Amiga, Inc., Addison-Wesley.

[4] Nintendo Co., Ltd. (1991). Super Nintendo Entertainment System Development Manual: Book I (PPU and Memory Architecture).

[5] Digital Display Working Group (DDWG). (1999). Digital Visual Interface (DVI) Specification Revision 1.0.

[6] JEDEC Solid State Technology Association. (2000). Synchronous DRAM Specification (JESD21-C).

[7] nand2mario. (2022). Gowin SDRAM Controller for Tang Nano 20K FPGA. GitHub Repository: https://github.com/nand2mario/nestang

[8] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is All You Need. Advances in Neural Information Processing Systems (NeurIPS 2017), pp. 5998–6008.

[9] Dettmers, T., Lewis, M., Belkada, Y., & Zettlemoyer, L. (2022). LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. In Advances in Neural Information Processing Systems (NeurIPS 2022).

[10] Gowin Semiconductor Corp. (2023). GW2AR Series of FPGA Products Data Sheet (DS226-1.9E).

[11] Sipeed. (2023). Tang Nano 20K Hardware Specification and Pinout Mapping. Sipeed Wiki Documentation.

[12] Espressif Systems. (2023). ESP32-S3 Technical Reference Manual (Version 1.4).

[13] VESA. (2006). VESA and Industry Standards and Guidelines for Computer Display Monitor Timing (DMT). Version 1.0, Rev. 11.

[14] Capello, D. (2016). Aseprite / LibreSprite File Format Specification (v1.1). https://github.com/aseprite/aseprite/blob/main/docs/ase-file-specs.md

[15] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. Advances in Neural Information Processing Systems (NeurIPS 2012), pp. 1097–1105.

APPENDIX - I

HARDWARE SYSTEM IMPLEMENTATION & PHYSICAL LAB PHOTOGRAPHS

This appendix documents physical photographs of the assembled hardware console prototype operating on the laboratory bench, interfacing the Sipeed Tang Nano 20K FPGA with the ESP32-S3 host computer, and driving 60 FPS video over physical HDMI.

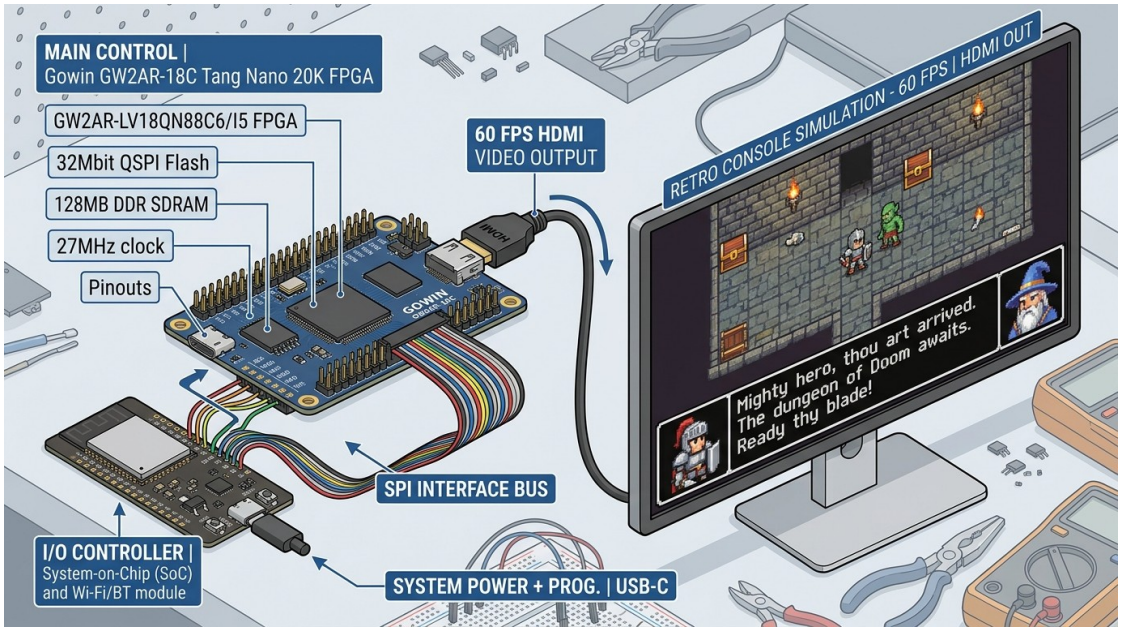


Figure A.1: Physical Hardware Integration Prototype: Tang Nano 20K FPGA & ESP32-S3 Host Controller on Breadboard with Active HDMI Video Output

APPENDIX - II

HARDWARE COMMAND PROTOCOL & PHYSICAL PINOUT CONSTRAINTS

This appendix details the physical FPGA pin constraint mappings configured in `constraints/tang_nano_20k.cst` for the Gowin GW2AR-LV18QN88C8/I7 package.

Table A.1: Physical FPGA Pinout & Signal Constraints (`tang_nano_20k.cst`)

Signal Identifier	FPGA Pin	ESP32-S3 Pin	I/O Standard	Hardware Signal Function
OSC_CLK	Pin 4	—	LVCMOS33	27.0 MHz On-Board Crystal Oscillator Input
RST_BTN	Pin 88	—	LVCMOS33 (Pull-up)	On-Board User Tactile Pushbutton (Active-Low)
SPI_CS_N	Pin 86	GPIO 10	LVCMOS33	Host SPI Chip Select (Active-Low)
SPI_SCLK	Pin 49	GPIO 12	LVCMOS33	Host SPI Master Clock (4.0 MHz, Mode 0)
SPI_MOSI	Pin 48	GPIO 11	LVCMOS33	Host SPI Master Out Slave In Data Line
SPI_MISO	Pin 51	GPIO 13	LVCMOS33	GPU Hardware Telemetry Return Line
HDMI_CLK_P / N	Pins 33 / 34	—	TLVDS_OBUF	DVI Differential Pixel Clock Pair (25.2 MHz)
HDMI_D0_P / N	Pins 35 / 36	—	TLVDS_OBUF	DVI Blue / Sync Differential Data Pair (126.0 MHz)

HDMI_D1_P / N	Pins 37 / 38	—	TLVDS_OBUF	DVI Green Differential Data Pair (126.0 MHz)
HDMI_D2_P / N	Pins 39 / 40	—	TLVDS_OBUF	DVI Red Differential Data Pair (126.0 MHz)
DIAG_LED[0:5]	Pins 15..20	—	LVCMOS33	Real-Time Diagnostic Status LEDs